🎮 Desenvolvimento de Jogo Web Interativo

"Adivinhe a Palavra" - Arquitetura e Implementação

2. Princípio de Responsabilidade Única

```
// Cada função tem uma única responsabilidade clara function showMessage(text, type) {
// Apenas gerencia exibição de mensagens } function updateDisplay() { // Apenas
atualiza a visualização } function guessLetter() { // Apenas gerencia a lógica de
adivinhação }
```

3. Separação de Responsabilidade

Camada	Responsabilidade	Tecnologia
Estrutura	Organização do conteúdo	HTML
Apresentação	Aparência visual	CSS
Comportamento	Interatividade e lógica	JavaScript

4. Estado Imutável vs Mutável

```
// Imutável - Não muda durante o jogo const words = [...]; // Array de palavras do
jogo // Mutável - Muda com cada ação let guessedLetters = []; // Estado do jogo atual
let attemptsLeft = 6; // Contador modificável
```

⚠ **Aviso:** Nunca modifique o array `words`. Se precisar adicionar palavras dinamicamente, crie uma cópia: `const newWords = [...words, newWord]`

🔧 Manipulação do DOM: Técnicas Avançadas

Métodos de Seleção de Elementos

```
// Por ID (mais rápido) document.getElementById('wordDisplay'); // Por classe (retorna coleção) document.getElementsByClassName('letter-box'); // Query selector (mais flexível) document.querySelector('.letter-box.revealed'); document.querySelectorAll('.letter-box');
```

Criar e Modificar Elementos

```
// 1. Criar elemento const box = document.createElement('div'); // 2. Configurar propriedades box.className = 'letter-box'; // Atribuir classe box.textContent = 'A'; // Texto do elemento box.id = 'letter-1'; // Atribuir ID // 3. Modificar classes dinamicamente box.classList.add('revealed'); // Adicionar classe box.classList.remove('hidden'); // Remover classe box.classList.toggle('active'); // Alternar on/off // 4. Inserir no DOM parent.appendChild(box); // Adicionar ao final parent.insertBefore(box, sibling); // Inserir antes de outro
```

Gerenciamento de Eventos

```
// Método 1: Inline HTML (usado no projeto) <button onclick="guessLetter()"> // Método 2: addEventListener (melhor prática) button.addEventListener('click', function() { guessLetter(); }); // Método 3: Arrow function (sintaxe moderna) button.addEventListener('click', () => guessLetter()); // Evento de teclado (implementado) input.addEventListener('keypress', (e) => { if (e.key === 'Enter') guessLetter(); });
```

💡 **Dica Pro:** Usa `addEventListener` em vez de atributos HTML inline para melhor separação de concerns e capacidade de adicionar múltiplos listeners ao mesmo evento.

Seção 7

🔄 Fluxo de Execução do Jogo

→ **1. CARREGAMENTO INICIAL**
O navegador carrega HTML → CSS → JavaScript
Se ejecuta `newGame()` automaticamente

→ **2. INICIALIZAÇÃO**

`newGame()` seleciona palavra aleatória
Cria array de traços: `['_', '_', '_', ...]`
Renderiza caixas vazias na tela

→ 3. INPUT DO USUÁRIO
Usuário digita uma letra → pressiona Enter ou clica
Evento é disparado → chama `guessLetter()`

→ 4. VALIDAÇÃO
É uma letra válida? → Já foi usada?
Se alguma validação falhar → mostra erro e termina

→ 5. PROCESSAMENTO
Adiciona letra a `usedLetters[]`
Está na palavra?
→ SIM: Revela letra(s) → Verifica vitória
→ NÃO: Reduz tentativas → Verifica derrota

→ 6. ATUALIZAÇÃO DA UI
`updateDisplay()` regenera todas as caixas
Aplica animaciones CSS automaticamente
Exibe mensagem de feedback

→ 7. CONDIÇÕES DE TÉRMINO
Vitória: Não restam traços → soma pontos → desabilita input
Derrota: Tentativas = 0 → revela palavra → desabilita input

Diagrama de Estado

```
INICIO ↓ [Novo Jogo] → Estado Inicial (attempts: 6, score: 0, guessed: [_, _, _]) ↓
[Jugando] ├─ Input válido │ ├─ Letra correta → Atualizar guessedLetters → Vitória? │ │
├─ Sí → [Game Won] │ │ └─ No → [Jugando] │ └─ Letra incorreta → Decrementar attempts →
¿Derrota? │ ├─ Sí → [Game Over] │ └─ No → [Jugando] └─ Input inválido → Mostrar erro →
[Jogando] [Game Won] / [Game Over] → Input deshabilitado → [Novo Jogo]
```

Seção 8

✔ Sistema de Validações

Validación en Camadas

```
function guessLetter() { const letter = input.value.toUpperCase(); // Camada 1: Validación de Existencia y Longitud if (!letter || letter.length !== 1) { showMessage('Digite apenas uma letra!', 'error'); return; } // Camada 2: Validación de Formato (RegEx) if (!letter.match(/[A-ZÁÀÂÃÉÊÍÓÔÕÚÇ]/)) { showMessage('Digite apenas letras!', 'error'); return; } // Camada 3: Validación de Estado (Duplicados) if (usedLetters.includes(letter)) { showMessage('Letra já usada!', 'error'); return; } // Se passar por todas as validações, processar // ... }
```

Expressões Regulares (RegEx)

Detalhamento de /[A-ZÁÀÂÃÉÊÍÓÔÕÚÇ]/

- `/` ... `/` = Delimitadores da expressão
- `[...]` = Set de caracteres (coincide com qualquer um dentro)
- `A-Z` = Letras maiúsculas A a Z
- `ÁÀÂÃÉÊÍÓÔÕÚÇ` = Vogais com acentos e Ç

```
// Outros exemplos úteis de RegEx: // Apenas números /[0-9]/ o /\d/ // Letras e números /[A-Za-z0-9]/ o /\w/ // Email (simplificado) /^[^\s@]+@[^\s@]+\.[^\s@]+$/ // Exatamente N caracteres /^.{5}$/ // Exatamente 5 caracteres
```

Validação HTML Nativa

```
<!-- Validações a nível HTML --> <input maxlength="1" <!-- Limita a 1 caractere --> required <!-- Campo obrigatório --> pattern="[A-Za-z]" <!-- Apenas letras --> type="text">
```

🎯 **Melhores Práticas:**

- Combine validação HTML + JavaScript (defesa em profundidade)
- Valide no cliente para UX, mas também no servidor para segurança
- Forneça mensagens de erro claras e específicas
- Use early returns para evitar aninhamento excessivo

Seção 9

⚡ Performance e Otimizações

1. Minimizar Reflows e Repaints

```
// ❌ Ruim - Múltiplas manipulações do DOM for (let i = 0; i < 100; i++) { const div =
```

```
document.createElement('div'); container.appendChild(div); // Reflow em cada iteração
} // ✅ Bom - Batch de operações const fragment = document.createDocumentFragment();
for (let i = 0; i < 100; i++) { const div = document.createElement('div');
fragment.appendChild(div); // Em memória, sem reflow }
container.appendChild(fragment); // Um único reflow
```

2. Delegação de Eventos

```
// ❌ Ruim - Event listener em cada elemento document.querySelectorAll('.letter-
box').forEach(box => { box.addEventListener('click', handleClick); }); // ✅ Bom -
Event listener no contêiner pai
document.getElementById('wordDisplay').addEventListener('click', (e) => { if
(e.target.classList.contains('letter-box')) { handleClick(e.target); } });
```

3. Debouncing e Throttling

```
// Throttle: Limita execuções a uma vez a cada X ms function throttle(func, delay) {
let lastCall = 0; return function(...args) { const now = Date.now(); if (now -
lastCall >= delay) { lastCall = now; func.apply(this, args); } }; } // Uso: Prevenir
spam de cliques button.addEventListener('click', throttle(guessLetter, 500));
```

4. Cache de Seletores

```
// ❌ Ruim - Buscar elemento repetidamente function update() {
document.getElementById('score').textContent = score;
document.getElementById('score').style.color = 'red'; } // ✅ Bom - Cachear referência
const scoreElement = document.getElementById('score'); function update() {
scoreElement.textContent = score; scoreElement.style.color = 'red'; }
```

Técnica	Benefício	Quando Usar
DocumentFragment	Reduz reflows	Inserir múltiplos elementos
Event Delegation	Menos memória	Muitos elementos similares
Throttle/Debounce	Controla frequência	Eventos rápidos (scroll, resize)
Caché de selectores	Evita buscas repetidas	Elementos usados frequentemente

Seção 10

🚀 Extensões e Melhorias Sugeridas

Feature 1: Sistema de Dificuldade

```
const difficulties = { easy: { attempts: 10, words: [/* palavras curtas */] }, medium: { attempts: 6, words: [/* palavras médias */] }, hard: { attempts: 4, words: [/* palavras longas */] } }; function setDifficulty(level) { attemptsLeft = difficulties[level].attempts; words = difficulties[level].words; }
```

Feature 2: LocalStorage para Persistência

```
// Salvar pontuação mais alta function saveHighScore() { const highScore = localStorage.getItem('highScore') || 0; if (score > highScore) { localStorage.setItem('highScore', score); } } // Carregar ao início function loadHighScore() { return localStorage.getItem('highScore') || 0; }
```

Feature 3: Categorias de Palavras

```
const wordsByCategory = { programming: [ { word: 'JAVASCRIPT', hint: 'Linguagem web' }, { word: 'PYTHON', hint: 'Serpente programadora' } ], animals: [ { word: 'ELEFANTE', hint: 'Mamífero grande' }, { word: 'JIRAFA', hint: 'Pescoço longo' } ] }; function selectCategory(category) { words = wordsByCategory[category]; newGame(); }
```

Feature 4: Temporizador

```
let timeLeft = 60; let timerInterval; function startTimer() { timerInterval = setInterval(() => { timeLeft--; document.getElementById('timer').textContent = timeLeft; if (timeLeft === 0) { clearInterval(timerInterval); gameOver(); } }, 1000); }
```

Feature 5: Animações Avançadas

```
// Animação quando uma letra é revelada function revealLetter(index) { const box = document.querySelectorAll('.letter-box')[index]; box.style.animation = 'pop 0.5s ease'; } // CSS para a animação @keyframes pop { 0% { transform: scale(1); } 50% { transform: scale(1.3); } 100% { transform: scale(1); } }
```

🔋 Modo Multijogador

Implemente WebSockets para jogar contra outros jogadores em tempo real

🎨 Temas Personalizáveis

Permita ao usuário alterar cores e aparência do jogo

🔊 Efeitos Sonoros

Adicione feedback auditivo com Web Audio API

📊 Estatísticas

Rastreie palavras adivinhadas, sequência de vitórias, tempo médio

Seção 11

🐛 Debugging e Ferramentas de Desenvolvimento

Métodos do Console

```
// Log básico console.log('Letra adivinhada:', letter); // Avisos console.warn('Poucas tentativas restantes!'); // Erros console.error('Validação falhou'); // Tabela (útil para arrays/objetos) console.table(guessedLetters); // Tempo de execução console.time('updateDisplay'); updateDisplay(); console.timeEnd('updateDisplay'); // Agrupar logs relacionados console.group('Validação de Letra'); console.log('Letra:', letter); console.log('Válida:', isValid); console.groupEnd();
```

🎯 Objetivos de Aprendizado

- ✓ **Compreender a arquitetura de uma aplicação web SPA (Single Page Application)**
- ✓ **Dominar a manipulação do DOM com JavaScript vanilla**
- ✓ **Implementar lógica de jogo com gerenciamento de estado**
- ✓ **Aplicar princípios de design UI/UX moderno**
- ✓ **Desenvolver código mantível e escalável**

Seção 1

🏗 Arquitetura da Aplicação

CAMADA DE APRESENTAÇÃO

HTML + CSS
Estrutura semântica e design visual responsivo

CAMADA DE LÓGICA

JavaScript
Gerenciamento de estado, eventos e validações

CAMADA DE DADOS

Variáveis em memória
Estado do jogo e configuração

📊 Modelo de Dados

```
// Estado Global da Aplicação const words = [ { word: 'JAVASCRIPT', hint: 'Linguagem de programação' } ]; // Estado do Jogo (Mutável) let currentWord = ''; // Palavra ativa let guessedLetters = []; // Letras adivinhadas let usedLetters = []; // Letras tentadas let attemptsLeft = 6; // Contador de tentativas let score = 0; // Pontuação
```

💡 **Design do Estado:** Separamos dados imutáveis (palavras do jogo) de dados mutáveis (estado atual). Isso facilita o reset do jogo e evita mutações acidentais da configuração.

Seção 2

📄 Estrutura HTML Semântica

Princípios de Design HTML

- **Semântica:** Uso correto de elementos HTML5
- **Acessibilidade:** IDs únicos para manipulação com JS
- **Separação de concerns:** Estrutura separada de apresentação e comportamento
- **Modularidade:** Componentes reutilizáveis

```
<!-- Contêiner Principal --> <div class="container"> <!-- Informações do Jogo --> <div
```

```
class="game-info"> <div id="attempts">6</div> <div id="score">0</div> </div> <!-- Display Dinâmico (Gerado com JS) --> <div id="wordDisplay"></div> <!-- Interação do Usuário --> <input id="letterInput" maxlength="1"> <button onclick="guessLetter()">Tentar</button> </div>
```

🔑 **Ponto Chave:** Os elementos com `id` são pontos de ancoragem para JavaScript. Usamos classes para estilos e IDs para manipulação do DOM.

Elemento	Propósito	Manipulação JS
`#wordDisplay`	Contêiner de letras	Renderização dinâmica
`#letterInput`	Entrada do usuário	Captura de eventos
`#message`	Feedback visual	Toggle de classes CSS
`#attempts`	Contador de tentativas	Atualização de texto

Seção 3

🎨 Sistema de Design CSS

1. Reset CSS e Box Model

```
* { margin: 0; padding: 0; box-sizing: border-box; /* Inclui padding/border no width */ }
```

Por que box-sizing: border-box?
Sem isso: `width: 200px + padding: 20px = 240px total`
Com isso: `width: 200px (inclui padding) = 200px total`

2. Layout com Flexbox

```
body { display: flex; justify-content: center; /* Centralizado horizontal */ align-items: center; /* Centralizado vertical */ min-height: 100vh; /* Altura completa viewport */ } .word-display { display: flex; gap: 10px; /* Espaçamento entre elementos */ flex-wrap: wrap; /* Quebra responsiva */ }
```

3. Gradientes Modernos

```
body { background: linear-gradient(135deg, #667eea 0%, #764ba2 100%); /* 135deg =
```

```
diagonal, #667eea = azul, #764ba2 = roxo */ }
```

4. Transições e Animações

```
button { transition: all 0.3s ease; /* Transição suave de todas as propriedades */ }
button:hover { transform: translateY(-2px); /* Efeito de elevação */ box-shadow: 0 5px
15px rgba(102, 126, 234, 0.4); } /* Animação keyframes */ @keyframes slideIn { from {
opacity: 0; transform: translateY(-20px); } to { opacity: 1; transform: translateY(0);
} }
```

✅ Variáveis CSS

Considere usar CSS custom properties para cores reutilizáveis:
`--primary: #667eea;`

✅ Mobile First

Projete primeiro para mobile, depois expanda com media queries

✅ Acessibilidade

Mantenha contraste mínimo 4.5:1 para texto

Seção 4

⚙ JavaScript: Lógica do Jogo

Função 1: newGame() - Inicialização

```
function newGame() { // 1. Seleção aleatória de palavra const randomIndex =
Math.floor(Math.random() * words.length); currentWord = words[randomIndex].word; // 2.
Inicializar array de letras adivinhadas com traços guessedLetters =
Array(currentWord.length).fill('_'); // 3. Reset do estado usedLetters = [];
attemptsLeft = 6; // 4. Atualizar UI document.getElementById('hint').textContent =
currentHint; updateDisplay(); }
```

🔍 Análise Técnica:

- `Math.random()` gera número entre 0 e 1
- Multiplicar por `words.length` dá range 0 a N
- `Math.floor()` arredonda para baixo para obter índice válido
- `Array().fill()` cria array com valores iniciais idênticos

Função 2: updateDisplay() - Renderização

```
function updateDisplay() { const wordDisplay = document.getElementById('wordDisplay');
wordDisplay.innerHTML = ''; // Limpar conteúdo anterior // Iterar sobre cada letra
guessedLetters.forEach((letter, index) => { // Criar elemento DOM dinamicamente const
box = document.createElement('div'); box.className = 'letter-box'; // Aplicar classe
condicional if (letter !== '_') { box.classList.add('revealed'); } box.textContent =
letter; wordDisplay.appendChild(box); }); }
```

⚡ **Otimização:** Em vez de modificar o DOM elemento por elemento, criamos todos os elementos e os inserimos em uma única operação. Para listas muito grandes, considere usar `DocumentFragment`.

Função 3: guessLetter() - Lógica Principal

```
function guessLetter() { const letter = input.value.toUpperCase(); // Validações em
cascata if (!letter || letter.length !== 1) { showMessage('Digite apenas uma letra!',
'error'); return; // Padrão early return } if (usedLetters.includes(letter)) {
showMessage('Letra já usada!', 'error'); return; } // Registrar letra usada
usedLetters.push(letter); // Verificar se a letra está na palavra if
(currentWord.includes(letter)) { // Revelar todas as ocorrências for (let i = 0; i <
currentWord.length; i++) { if (currentWord[i] === letter) { guessedLetters[i] =
letter; } } // Verificar vitória if (!guessedLetters.includes('_')) { score +=
attemptsLeft * 10; showMessage('¡Ganaste! 🏆', 'success'); } } else { // Letra
incorreta attemptsLeft--; if (attemptsLeft === 0) { showMessage(`Game Over! Palabra:
${currentWord}`, 'error'); } } updateDisplay(); }
```

Seção 5

🎨 Padrões de Design Aplicados

1. Padrão Early Return

```
// ❌ Ruim - Aninhamento profundo function validate(input) { if (input) { if
(input.length > 0) { if (input.match(/[A-Z]/)) { // Processar... } } } } // ✅ Bom -
Early returns function validate(input) { if (!input) return false; if (input.length
=== 0) return false; if (!input.match(/[A-Z]/)) return false; // Processar... }
```